FOODVERSE AI

Multimodal Food Intelligence Agent

PROJECT DESIGN PHASE

System architecture, component design, API flow, and interface design

1. Design Objectives

The design phase translates the Foodverse AI concept and requirements into a modular application structure. The design aims to keep the user interface, API routes, feature logic, and external AI integrations separated so that individual features can be developed, tested, and improved without unnecessarily affecting the rest of the application.

2. High-Level Architecture

Foodverse AI follows a web-application pattern: users interact with a frontend; the frontend sends requests to a backend API; the backend validates and processes the request; and a structured response is returned for display. The exact implementation of each feature depends on the code and services configured in the project.

Frontend Forms, image selection where supported, results and error messages
HTTP/API layer Request routing, input validation, status codes and JSON responses
Feature modules Recipe generation, food recognition, and other implemented food-related functions
AI/model or external services (optional/configuration-dependent) Image models or language-model services when configured

3. Main Components

3.1 Frontend

The frontend is responsible for presenting the available Foodverse AI features, collecting the user's inputs, sending requests to the correct API endpoint, and displaying results. It should show loading states and clear messages when an input is invalid or the API cannot be reached. Food recognition UI should only be described as complete if the corresponding component and endpoint have been integrated.

3.2 Backend API

The backend is implemented using Python and FastAPI, with Uvicorn used to run the application during development. API routes separate individual capabilities and make them testable independently. The FastAPI documentation page at /docs supports interactive testing while the backend is running.

3.3 Feature modules

- **Recipe generation:** receives ingredients and supported constraints, such as dietary restriction and maximum preparation time.
- **Food recognition:** accepts supported image input and returns a structured result when the feature is implemented.
- **Freshness-related support:** provides preliminary guidance only, if this module is available.
- **Hygiene-related support:** provides structured observations or scores only where implemented and validated.

3.4 AI and external integrations

Model libraries and external service packages may be used for specific tasks. The design should isolate such integrations behind clear functions or service modules, read credentials from environment variables, and return controlled errors when a service is unavailable. The presence of a library alone does not mean that a live integration is active.

4. Conceptual Request Flow

- The user chooses a feature and enters the required information.
- The frontend validates basic form requirements and sends the request to the backend.
- FastAPI routes the request to the appropriate endpoint.

- The endpoint validates the data and calls the relevant feature logic or configured model/service.
- The backend returns a structured success response or a meaningful error response.
- The frontend presents the result to the user.

5. Example: Recipe Generation Flow

Example request values discussed during development:

- Ingredients: paneer, spinach, cherry tomatoes.
- Dietary restriction: vegetarian.
- Maximum preparation time: 30 minutes.

The frontend submits these values to the recipe-generation endpoint (discussed as POST /api/v1/recipes/generate). The backend validates the request, runs the available recipe-generation logic, and returns a structured response. The precise response fields depend on the implemented API schema.

6. API Design Principles

- Use clear endpoint names and HTTP methods that match the operation.
- Validate required values and file uploads before processing.
- Return consistent JSON structures for successful API calls.
- Use appropriate HTTP status codes and useful error messages.
- Keep route handlers small and move reusable logic into separate functions or modules where practical.
- Avoid returning secrets, stack traces, or sensitive configuration to users.

7. Data and File Handling

Recipe requests use structured fields such as ingredients and constraints. Image-based endpoints, where implemented, may use multipart form uploads. Uploaded images should be checked for presence, supported format, and reasonable size before processing. The application should avoid storing user uploads longer than necessary unless storage is an explicit requirement.

8. Error Handling and Security Design

- Handle missing fields, empty values, invalid files, and unsupported formats gracefully.
- Use environment variables for API keys and deployment-specific settings.
- Do not commit .env files or credentials to the GitHub repository.
- Configure frontend-to-backend access appropriately when deployed, including CORS where required.
- Treat freshness and hygiene results as advisory and avoid safety guarantees based solely on images.

9. Deployment-Oriented Design Considerations

The frontend and backend may be hosted separately. In that case, the frontend must use the deployed backend URL rather than a local address such as localhost. The backend host must install the required dependencies, use the correct application start command, and receive any required environment variables. The final hosting configuration should be tested using the public URL.

10. Design Decisions and Trade-offs

- **Modular APIs:** make features easier to test and maintain, though they require consistent interfaces between modules.

- **Separate frontend and backend:** supports independent deployment and development, but requires correct URL and CORS configuration.
- **Optional AI integrations:** allow gradual implementation and reduce tight coupling, but features depending on external services may fail when keys or network access are unavailable.
- **Structured responses:** simplify frontend integration, but response schemas should remain consistent as the project evolves.

11. Design Deliverables

- A high-level architecture separating frontend, API layer, feature modules, and optional AI services.
- A conceptual request flow from user input to displayed result.
- An API design approach with validation, structured responses, and error handling.
- Security and deployment considerations to guide implementation and testing.